



Instituto Politécnico Nacional



upiita-ipn

Unidad Profesional Interdisciplinaria en Ingeniería
Y Tecnologías Avanzadas

Bases de Datos Distribuidas

“Arquitectura y tipo de gestores”

Alumnos:

De la Rosa Valdes Itztli Fernanda

Luviano Moreno Alondra

Peña Armas Karina Michelle

1. Arquitecturas de Sistemas de Bases de Datos Distribuidas (DDBMS)

Un Sistema Manejador de Bases de Datos Distribuidas (DDBMS) es un conjunto integrado de software que administra múltiples bases de datos lógicamente interrelacionadas, las cuales se encuentran distribuidas geográficamente a lo largo de una red de comunicaciones, proveyendo al usuario una interfaz unificada y transparente.

A diferencia de un entorno centralizado, el diseño arquitectónico de un DDBMS debe equilibrar tres dimensiones fundamentales:

- **Autonomía:** El grado en que los nodos individuales pueden operar independientemente y tomar decisiones sobre sus recursos locales.
- **Distribución:** La dispersión física de los componentes, datos y procesamiento a través de múltiples sitios interconectados.
- **Heterogeneidad:** La disparidad existente entre el hardware, sistemas operativos, protocolos de red y modelos o gestores de bases de datos locales (DBMS).

1.1. Taxonomía Arquitectónica

A. Arquitectura Cliente-Servidor

Separa claramente las responsabilidades del procesamiento en dos entidades lógicas:

- **Cliente:** Administra la interfaz de usuario, valida entradas y gestiona la lógica de presentación/aplicación. Envía consultas y solicitudes de transacciones al servidor a través de la red.
- **Servidor:** Responsable de la gestión del almacenamiento de datos, ejecución del motor transaccional, optimización de consultas, control de concurrencia y mecanismos de recuperación.

- **Variantes:**

- Cliente-Servidor 2 Niveles: El cliente aloja la aplicación completa y se comunica directamente con el servidor de datos.
- Cliente-Servidor Multi-Nivel (3 o N niveles): Se desacopla la lógica de negocio en servidores de aplicaciones intermedias, mejorando la escalabilidad y seguridad.

B. Arquitectura Par a Par (Peer-to-Peer / P2P)

- En este paradigma no existe una jerarquía estricta maestro/esclavo ni una división fija entre emisor y receptor.
- Cada nodo (o sitio de la red) posee capacidades idénticas: actúa simultáneamente como cliente (generando transacciones globales) y como servidor (procesando consultas y almacenando datos locales).
- Proporciona alta tolerancia a fallos y elimina cuellos de botella centralizados, aunque incrementa sustancialmente la complejidad en los protocolos de consenso, bloqueo distribuido y detección de interbloqueos (deadlocks).

C. Sistemas Homogéneos vs. Heterogéneos

- **Sistemas Homogéneos:** Todos los sitios ejecutan el mismo motor de base de datos (mismo software y versión). Comparten el mismo modelo de datos y esquemas lógicos. Facilitan la optimización de consultas distribuidas y el control de concurrencia.
- **Sistemas Heterogéneos (Multidatabase / Federados):** Integran distintos gestores (por ejemplo, nodos bajo Oracle, otros con PostgreSQL y otros con motores NoSQL). Requieren capas de mediación (*gateways*, *wrappers* y traductores de esquemas) para homologar los lenguajes de consulta y las transacciones entre plataformas heterogéneas.

1.2. Modelos de Organización

- **Cliente-Servidor (2 y múltiples niveles):** Los clientes gestionan la interfaz y lógica de aplicación; los servidores ejecutan las funciones de almacenamiento, control de concurrencia y procesamiento de consultas.
- **Par a Par (Peer-to-Peer):** Todos los sitios cuentan con la misma jerarquía funcional; cada nodo actúa tanto de cliente emisor como de servidor local para el resto del clúster.
- **Sistemas Homogéneos vs. Heterogéneos (Multidatabase / MDBS):** Los homogéneos comparten el mismo software y esquema en todos los nodos; los heterogéneos integran DBMSs preexistentes y autónomos mediante un esquema global mediador (federados).

1.3. Esquema Arquitectónico ANSI/SPARC Generalizado

La arquitectura por niveles desacopla la percepción global del usuario de la implementación en hardware local:

- **Esquema Conceptual Global (GCS):** Visión integrada de los datos como una sola entidad lógica.
- **Esquema de Fragmentación (FS):** Reglas formales para dividir cada relación global en fragmentos disjuntos.
- **Esquema de Asignación / Ubicación (AS):** Mapeo que asigna cada fragmento a un nodo físico concreto (incluyendo políticas de replicación).
- **Esquema Conceptual Local (LCS) y Esquema Interno Local (LIS):** Definición física del almacenamiento y procesamiento en el DBMS de cada sitio particular.

1.4. Reglas Formales de Fragmentación

Toda fragmentación de una relación R en subconjuntos $R_1, R_2, \dots, R_n$ debe satisfacer tres propiedades axiomáticas según el libro:

1. **Compleitud:** Ningún dato se pierde. Toda tupla o atributo de R pertenece a al menos un R_i .
 2. **Reconstrucción:** Existe un operador relacional que regenera R intacta.
 3. **Disyunción:** No existe redundancia innecesaria. Para horizontal, los fragmentos son mutuamente excluyentes; para vertical, solo la clave primaria se repite para reconstrucción.
- **Tipos:** Horizontal (primaria y derivada), Vertical (por proyección de columnas) y Mixta/Híbrida.

1.5. Grados de Replicación y Formas de Transparencia

- **Replicación:** Puede ser *Completa* (todos los nodos tienen toda la base de datos; maximiza lectura pero encarece escrituras), *Parcial* (se replican solo fragmentos de alta demanda) o *Nula* (particionamiento puro sin redundancia).
- **Transparencia (Ocultamiento de complejidades):**
 - De ubicación: Desconocer en qué nodo reside el dato.
 - De fragmentación: Consultar entidades sin conocer cómo están divididas.
 - De replicación: Ignorar la existencia de múltiples copias físicas.
 - De concurrencia y fallos: Garantizar atomicidad y aislamiento transaccional frente a fallas parciales de enlaces o nodos.

2. Tipos y Análisis de Gestores de Bases de Datos Distribuidas

Gestor	Modelo de Datos	Estrategia de Distribución	Soporte Técnico (Fragmentación y Replicación)
MySQL (NDB)	Relacional	Fragmentación y Replicación	<i>Sharding</i> horizontal automático y replicación síncrona/asíncrona.
SQL Server	Relacional	Fragmentación y Replicación	Particionamiento de tablas por rangos y grupos <i>Always On</i> .
Oracle DB	Relacional	Fragmentación y Replicación	<i>Oracle Sharding</i> nativo y replicación activa (<i>Data Guard</i>).
Google Spanner	Relacional (NewSQL)	Fragmentación y Replicación	Particionado dinámico automático y réplica síncrona por Paxos.
MongoDB	NoSQL (Documentos)	Fragmentación y Replicación	<i>Sharding</i> por rangos o hash y sincronización vía <i>Replica Sets</i> .
Cassandra	NoSQL (Columnas)	Fragmentación y Replicación	Particionado por hash en anillo y factor de replicación ajustable.
Redis	NoSQL (Clave-valor)	Fragmentación y Replicación	Particionado en 16,384 ranuras y réplica primaria-secundaria.
Neo4j	NoSQL (Grafos)	Fragmentación y Replicación	Partición federada (<i>Fabric</i>) y consenso <i>Raft</i> en clúster causal.

3. Referencias Consultadas

- Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems* (4th ed.). Cham, Switzerland: Springer International Publishing.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.